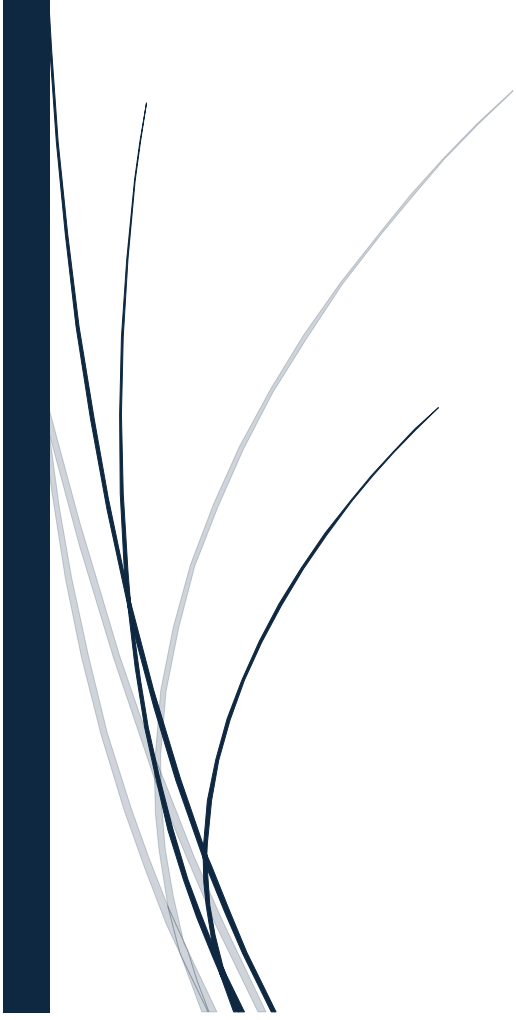
13/10/2026

SAE IA

Documentation des tests de modèles et des mesures



Enseignants :
DROUHIN Frédéric
REHM Guillaume

Groupe 3 :
MULLER Yoann
SOUARE Ibrahima
BRODIN Mathias

Table des matières

I. Conteneurisation 2
1) Objectif : 2
2) Architecture et configuration : 3
3) Déploiement et vérifications : 4
II. Banc d'essai des modèles (Benchmarking)* 5
1) Test des IA sur le sujet des mots de passe : 5
1.1. Contexte 5
1.2. Test avec le modèle llama3.1:8b 5
1.3. Test avec mistral:7b-instruct 7
1.4. Test avec le modèle phi3:mini 9
1.5. Conclusion générale 10
2) Mesures : 10
2.1. Méthodes 10
2.2. Résultats de performance 11
2.3. Résultats de qualité 13
2.4. Conclusion 14
2.5. Limites et améliorations 14
III. Prise en main de l'assistant OpenCode 15
1) Installation des pquets et des extensions nécessaires : 15
2) Génération de l'API calculatrice REST en Python/FastAPI : 15
2.1. On lance l'assistant OpenCode 15
2.2. On arrive sur cette page 15
2.3. On lui donne un prompt de ce type 16
3) Test/Vérification du fonctionnement de la calculatrice 16
On obtient 16
4) Test unitaires : 17
5) Analyse statique du code et scan de vulnérabilité : 17

I. Conteneurisation

1) Objectif :

L'objectif de cette étape est de déployer une infrastructure permettant d'exécuter des modèles de langage locaux (LLM, Large Language Models) et de les utiliser à travers une interface web.
Pour cela, deux services sont déployés via le fichier docker-compose.yml :

- Ollama : assure le téléchargement, le stockage et l'exécution des modèles de langage.
- Open WebUI : fournit une interface graphique accessible depuis un navigateur web pour interagir avec les modèles hébergés par Ollama.

La conteneurisation permet d'isoler les services, de simplifier leur déploiement et de faciliter la gestion de leur configuration. Elle permet également de conserver les données importantes entre les redémarrages des conteneurs grâce aux volumes persistants.

2) Architecture et configuration :

```yaml
services:
  ollama:
    image: ollama/ollama:latest
    container_name: ollama
    restart: unless-stopped
    ports:
      - "11435:11434"
    volumes:
      - ollama_data:/root/.ollama
    environment:
      OLLAMA_HOST: "0.0.0.0:11434"
  open-webui:
    image: ghcr.io/open-webui/open-webui:main
    container_name: open-webui
    restart: unless-stopped
    ports:
      - "3000:8080"
    volumes:
      - open_webui_data:/app/backend/data
    environment:
      OLLAMA_BASE_URL: "http://ollama:11434"
    depends_on:
      - ollama
volumes:
  ollama_data:
  open_webui_data:
```

Le fichier docker-compose.yml définit les deux conteneurs :

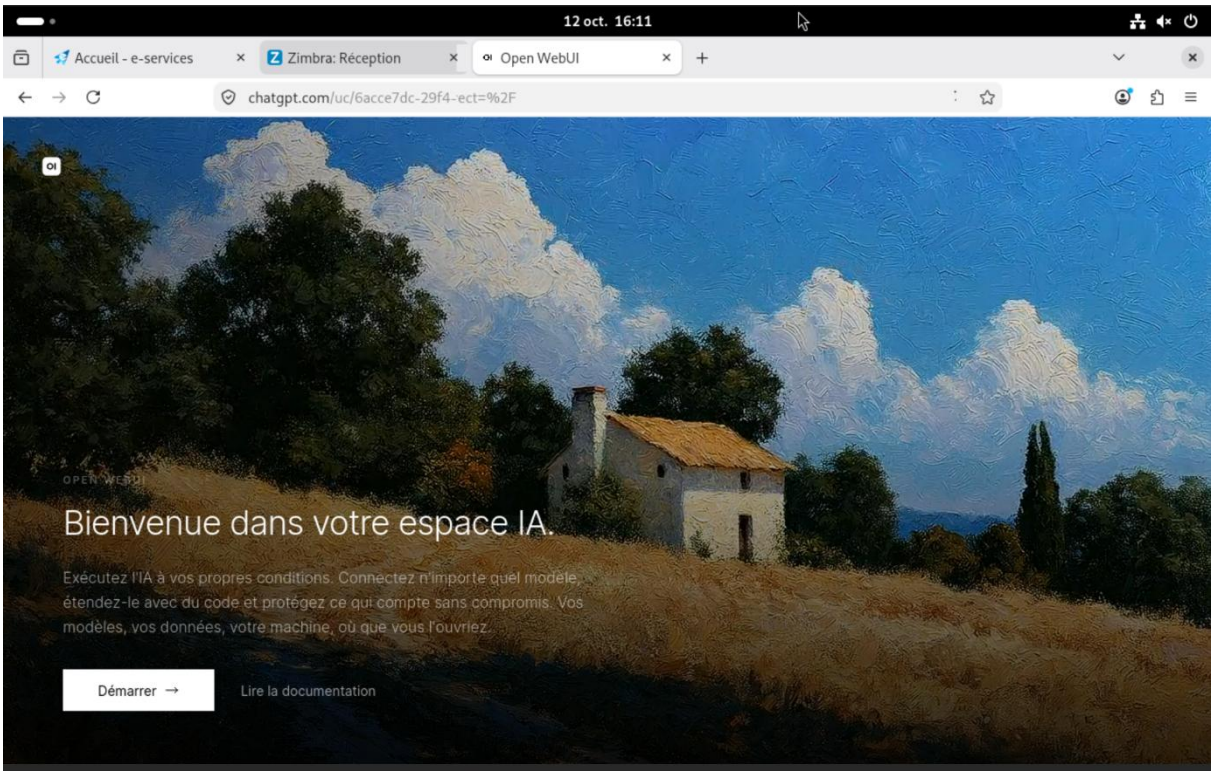
- Ollama : exécute les modèles et expose son API HTTP sur le port 11434.
- Open WebUI : fournit une interface accessible sur http://localhost:3000 et communique avec Ollama via le réseau interne Docker.

Les volumes ollama_data et open_webui_data assurent la persistance des modèles, des conversations et des paramètres, même après la recréation des conteneurs.

La ligne restart: unless-stopped permet le redémarrage automatique des services et depends_on définit leur ordre de démarrage.

3) Déploiement et vérifications :

Depuis le répertoire contenant le fichier docker-compose.yml :
docker compose up -d
Puis on peut tester Ollama avec :
curl http://localhost:11435/api/tags
Enfin on peut accéder à l'interfcae Open WebUI via l'adresse http://localhost:3000.



II. Banc d'essai des modèles (Benchmarking)*

1) Test des IA sur le sujet des mots de passe :

1.1. Contexte

On donne au 3 modèles d'ia des critères de mot de passes à respecter et on leur donne 2 mots de passe différents : le premier valide tous les critères imposés contrairement au 2ème mot de passe où il manque le caractère spécial

Les critères utilisés pour ce test sont les suivants :

-12 caractères minimum

-Au moins 1 majuscule

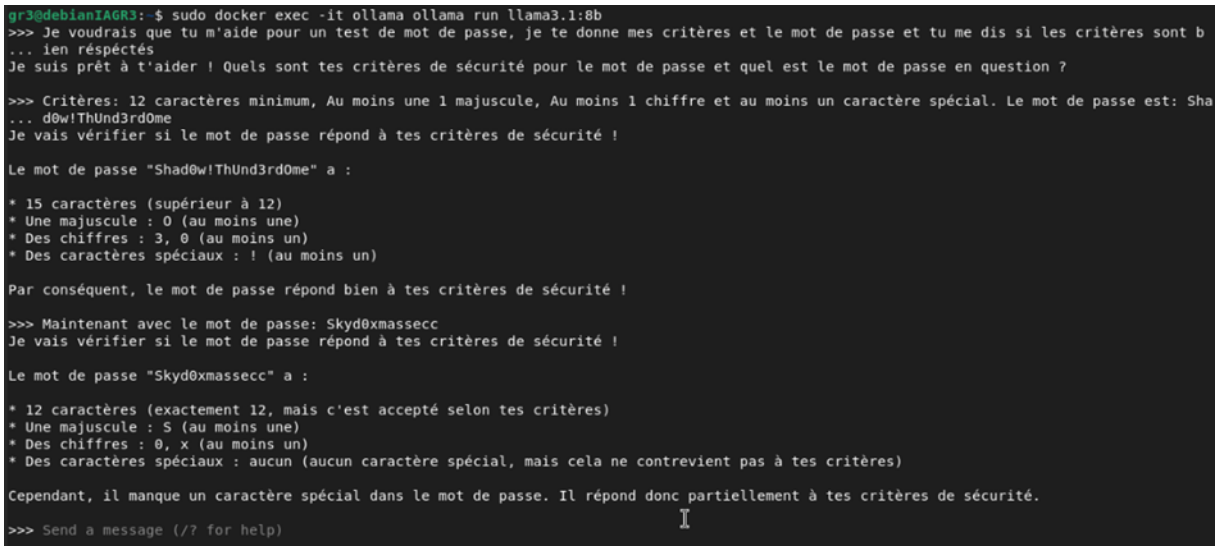
-Au moins 1 chiffre

-Au moins 1 caractères spéciaux (! , ? , #, . , ect)

Et les mots de passe utilisés pour ce test sont Shad0w!ThUnderdOme et Skyd0xmassecc

On a fait ces tests sur les modèles suivants : llama3.1:8b, mistral:7b-instruct et phi3:mini

1.2. Test avec le modèle llama3.1:8b



Bons points :

-Il produit une réponse claire et relativement courte.

-Il classe correctement le premier mot de passe comme conforme.

-Il repère directement que Skyd0xmassecc ne contient pas de caractère spécial.

-Il indique finalement que le deuxième mot de passe ne répond que partiellement aux critères.

Mauvais points :

-Il donne une longueur incorrecte pour les deux mots de passe.

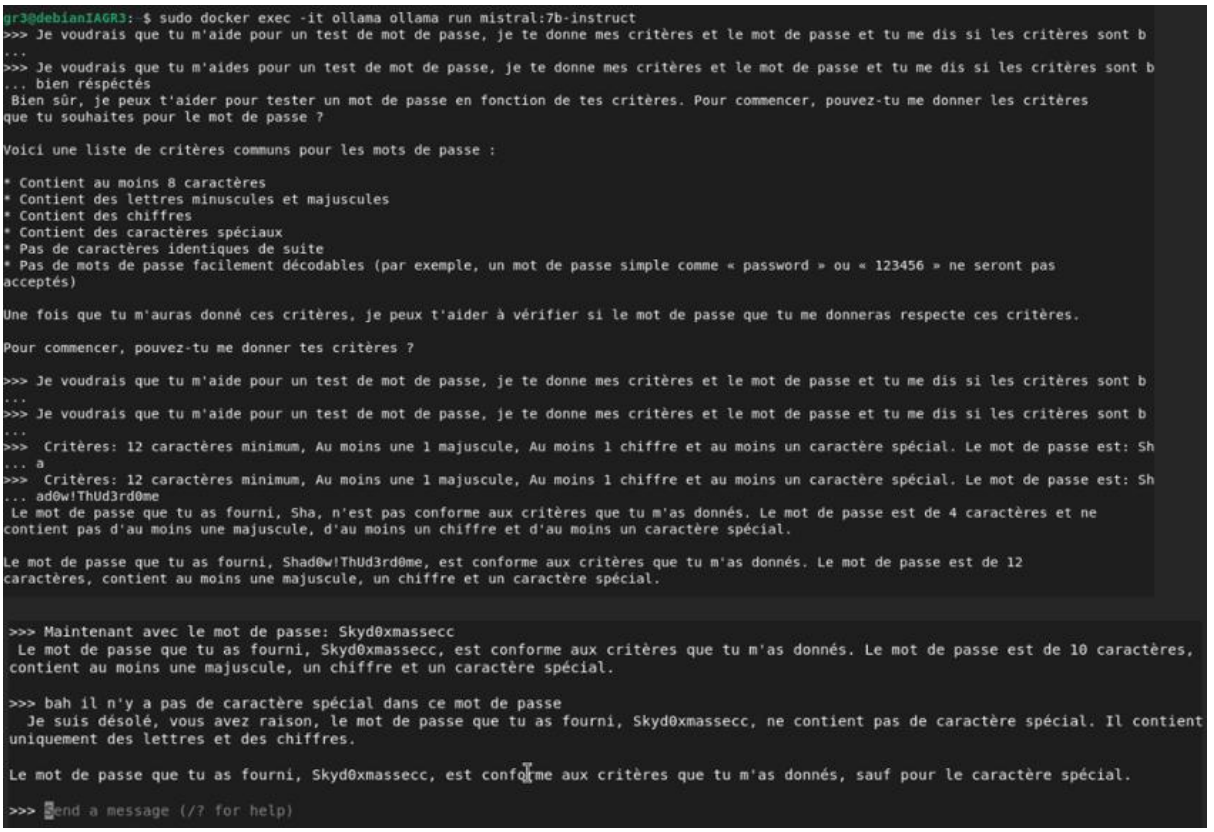
-Son calcul des chiffres et des majuscules du premier mot de passe est incorrect.

-Pour le deuxième mot de passe, il écrit d'abord que l'absence de caractère spécial « ne contrevient pas » aux critères, alors que ce caractère est explicitement obligatoire. Et ensuite il affirme que le mot de passe répond « partiellement » aux critères, ce qui contredit sa phrase précédente.

Conclusion sur le modèle Llama :

Llama 3.1 8B comprend globalement la consigne et repère la principale erreur du second mot de passe. Ses réponses sont lisibles et directes, mais elles comportent des erreurs de calcul ainsi qu'une contradiction importante : le modèle reconnaît l'absence de caractère spécial tout en indiquant que cela n'est pas contraire aux critères. Son verdict final est juste, mais son raisonnement manque de rigueur.

1.3. Test avec mistral:7b-instruct



Bons Points :

-Il répond de manière concise.

-Il classe finalement correctement le premier mot de passe comme conforme.

-Après correction par l'utilisateur, il reconnaît que le deuxième mot de passe ne contient pas de caractère spécial.

-Il explique la raison de la non-conformité après avoir été repris.

Mauvais points :

-Il se trompe sur le compte de la longueur des mots de passe.

-Il affirme initialement que Skyd0xmassecc contient un caractère spécial alors que ce n'est pas le cas (Il valide à tort le deuxième mot de passe).

-La correction ne vient pas de sa propre vérification : elle intervient uniquement après le signalement de l'utilisateur.

Conclusion sur le modèle Mistral :

Mistral 7B Instruct est un modèle pas vraiment fiable. Même si sa réponse est concise, il ne détecte pas spontanément l'absence de caractère spécial dans le second mot de passe et donne d'abord un verdict incorrect. Il est capable de se corriger mais que lorsque l'utilisateur lui montre son erreur ce qui crée une grande dépendance et montre une très faible fiabilité.

1.4. Test avec le modèle phi3:mini



Bons points :

-Il classe correctement le premier mot de passe comme conforme.

-Il détecte immédiatement l'absence de caractère spécial dans le second mot de passe.

-Il conclut correctement que Skyd0xmassecc est non conforme.

-Sa réponse est bien organisée sous forme de critères numérotés.

Mauvais points :

-Il se trompe pour le nombre de caractères pour les 2 mots de passe : il annonce 20 caractères au lieu de 18 pour le premier et 16 caractères au lieu de 13 pour le deuxième.

-Il compte deux chiffres, alors que Shad0w!ThUnderdOme n'en contient qu'un : le 0.

-Il ajoute des critères que l'on n'avait pas demandés.

Conclusion Phi-3 :

Phi-3 Mini est un modèle performant. Il comprend correctement la logique de validation et donne les deux bons verdicts dès sa première analyse. Sa réponse est détaillée, structurée et facile à suivre. Cependant, ses erreurs montrent que son analyse n'est pas totalement fiable lorsqu'il doit examiner précisément chaque caractère d'une chaîne.

1.5. Conclusion générale

Les trois modèles comprennent globalement le principe de vérification d'un mot de passe à partir de critères imposés. Phi-3 Mini se démarque par une analyse plus complète, mieux structurée et par la détection immédiate de la non-conformité du second mot de passe. Llama 3.1 8B repère l'erreur principale, mais formule une conclusion contradictoire.

Mistral 7B Instruct obtient le test le moins satisfaisant, car il valide initialement un mot de passe non conforme et ne se corrige qu'après l'intervention de l'utilisateur.

2) Mesures :

2.1. Méthodes

1. Mesures de vitesse : script bench.sh , API d'Ollama.

2. Un appel de chauffe par modèle (chargement en RAM, non compté).

3. 3 mesures par modèle, même prompt pour les 3 : « Expliques-en 5 phrases ce qu'est un

Conteneur Docker. »

4. TTFT approché par : temps de chargement + temps d'analyse du prompt.

5. Débit : tokens générés divisés par le temps de génération.

6. CPU et RAM : docker stats sur le conteneur Ollama, une mesure toutes les 3 secondes environ, en parallèle du test.

7. Tout est lancé par run_all.sh, donc reproductible.

8. Qualité : script qualite.sh, 5 questions identiques, température à 0, notation de 0 à 2 par question (total sur 10).

2.2. Résultats de performance

```
gr3@debianIAGR3:~$ mkdir -p resultats_doc && cp bench.csv stats.csv bench_log.txt benchmark_modeles.pn
g resultats_doc/
./run_all.sh
10:45:09 >>> phi3:mini
10:45:57 >>> mistral:7b-instruct
10:47:21 >>> llama3.1:8b
10:49:00 modele,run,ttft_s,tokens_s
10:49:00 "phi3:mini","1",0.269991813,27.43936214071656
10:49:00 "phi3:mini","2",0.039523556,27.961347195747468
10:49:00 "phi3:mini","3",0.035987044,28.080804102084524
10:49:00 "mistral:7b-instruct","1",0.626544972,13.941051158832005
10:49:00 "mistral:7b-instruct","2",0.068792036,13.039909225269307
10:49:00 "mistral:7b-instruct","3",0.066695222,12.090257026556937
10:49:00 "llama3.1:8b","1",0.586595187,8.712603196251653
10:49:00 "llama3.1:8b","2",0.163451643,13.087840625565006
10:49:00 "llama3.1:8b","3",0.163159619,13.995122838715108
Termine : bench.csv, stats.csv, bench_log.txt
gr3@debianIAGR3:~$
```

```
gr3@debianIAGR3:~$ head -20 stats.csv
10:45:09,28.85%,92.87MiB / 62.8GiB
10:45:12,17.42%,356.4MiB / 62.8GiB
10:45:14,12.19%,559MiB / 62.8GiB
10:45:17,15.87%,822.7MiB / 62.8GiB
10:45:20,9.47%,1.017GiB / 62.8GiB
10:45:22,15.06%,1.256GiB / 62.8GiB
10:45:25,10.58%,1.437GiB / 62.8GiB
10:45:27,17.14%,1.729GiB / 62.8GiB
10:45:30,14.20%,1.959GiB / 62.8GiB
10:45:32,1394.33%,3.636GiB / 62.8GiB
10:45:35,1595.43%,3.635GiB / 62.8GiB
10:45:38,1598.20%,3.635GiB / 62.8GiB
10:45:40,1583.77%,3.635GiB / 62.8GiB
10:45:43,1579.49%,3.736GiB / 62.8GiB
10:45:45,1597.49%,3.735GiB / 62.8GiB
10:45:48,1592.22%,3.735GiB / 62.8GiB
10:45:50,1592.13%,3.815GiB / 62.8GiB
10:45:53,1591.12%,3.815GiB / 62.8GiB
10:45:56,1573.36%,3.815GiB / 62.8GiB
10:45:58,22.85%,276.3MiB / 62.8GiB
gr3@debianIAGR3:~$
```

Modèle	TTFT moyen	Tokens/s	RAM pic	CPU moyen
phi3:mini	0,10 s	26,1	3,99 Go	98 %
mistral:7b-instruct	0,17 s	14,6	4,89 Go	99 %
llama3.1:8b	0,26 s	13,7	5,47 Go	99 %

1. phi3:mini est le plus rapide (environ 80 % de tokens/s en plus) et le plus léger.

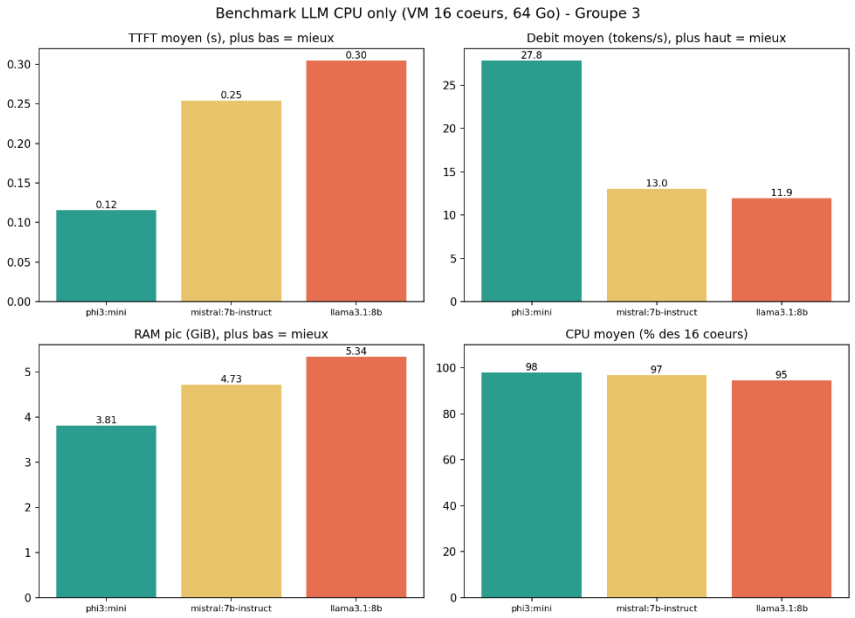
2. mistral et llama3.1 sont proches en vitesse, l'écart est dans la marge de mesure.

3. Le CPU est saturé pour les 3 modèles (environ 1600 % dans docker stats, soit les 16 cœurs). Ce critère ne les départage pas.

4. La RAM augmente avec la taille du modèle, loin des 64 Go disponibles.

5. Contrôle côté Proxmox : le pic CPU de la VM coïncide avec l'exécution du benchmark.

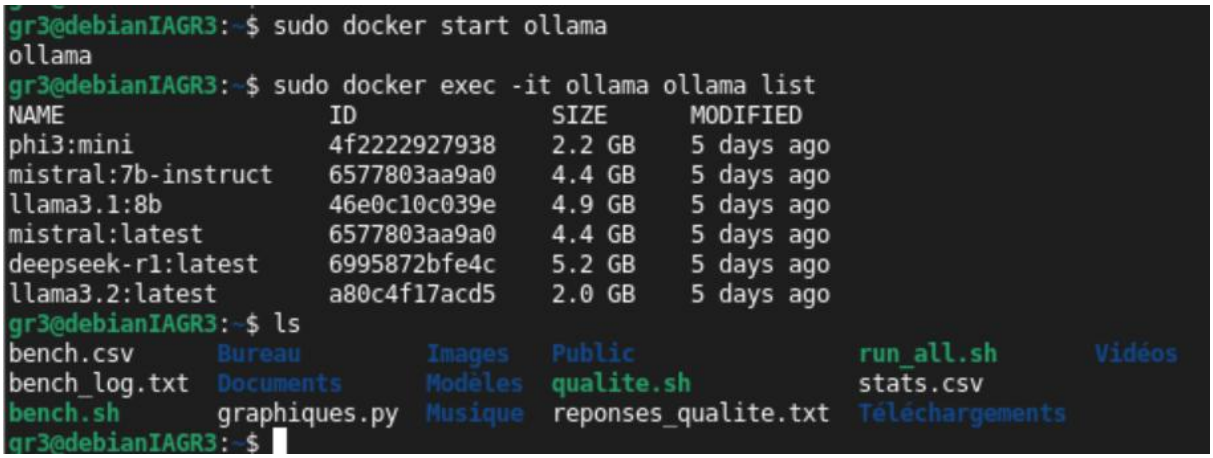
Graphiques comparatifs : voir benchmark_modeles.png.

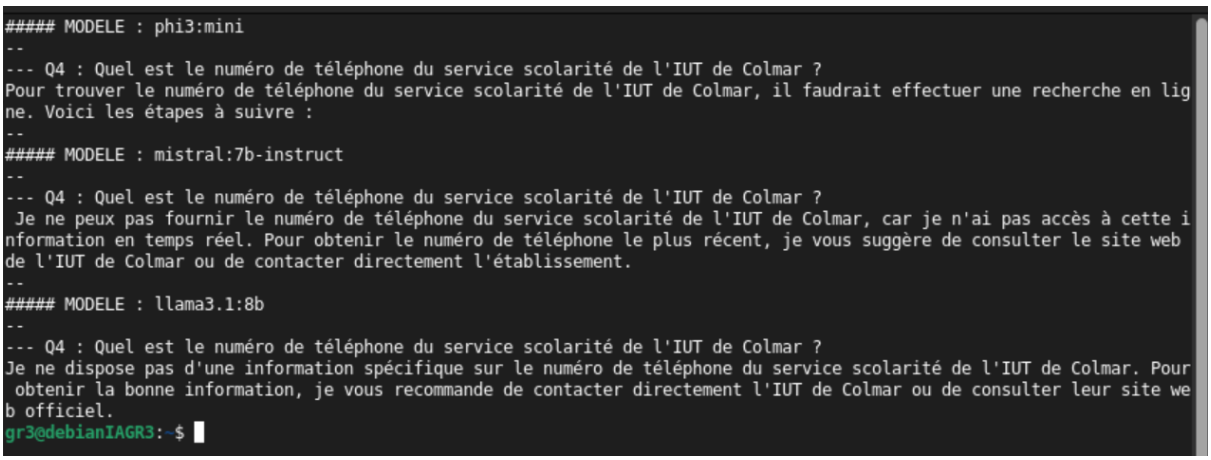


2.3. Résultats de qualité

Question	phi3:mini	mistral:7b	llama3.1:8b
Q1 VM contre conteneur	2	1	2
Q2 calcul mémoire	2	0	2
Q3 consigne stricte	1	2	2
Q4 piège hallucination	1	2	2
Q5 résumé	2	1	1
Total /10	**8**	**6**	**9**

1. mistral : erreur de calcul (11 au lieu de 12) et consignes de longueur non respectées.
2. phi3 : hors sujet sur Q3, et sur Q4 il n'indique pas qu'il ne sait pas.
3. llama3.1 : le plus fiable, mais invente la signification de RAG en Q5.
4. Aucun modèle n'invente de numéro de téléphone en Q4.





2.4. Conclusion

1. Modèle retenu : llama3.1:8b.

2. Meilleure note de qualité (9/10) et bon réflexe « je ne sais pas » face à une question sans réponse possible.

3. Coût acceptable : environ 14 tokens/s et 5,5 Go de RAM, très en dessous des 64 Go de la VM.

4. Choix assumé : pour un chatbot RAG évalué sur les hallucinations, la fiabilité prime sur la vitesse.

5. phi3:mini gardé en secours : le plus rapide, utile pour des tâches simples.

6. mistral:7b-instruct écarté : dernier en qualité (6/10) sans avantage de vitesse.

2.5. Limites et améliorations

1. TTFT approché, pas mesuré en streaming.

2. Un seul prompt de vitesse et 3 mesures par modèle : résultats à lisser avec plus de runs.

3. Qualité évaluée sur 5 questions, notation manuelle : à reconfirmer avec le RAG sur les documents du SIRT.

4. CPU et RAM échantillonnés avec docker stats. Un monitoring complet (Prometheus, cAdvisor, Grafana) serait plus précis.

5. Les métriques sur les modèles d'embeddings seront ajoutées avec le RAG.

III. Prise en main de l'assistant OpenCode

1) Installation des pquets et des extensions nécessaires :

sudo apt install npm

sudo npm install -g opencode-ai

sudo apt install python3 python3-pip python3-venv

python3 -m venv .venv

source .venv/bin/activate

pip install fastapi uvicorn pytest httpx

pip install bandit

2) Génération de l'API calculatrice REST en Python/FastAPI :

2.1. On lance l'assistant OpenCode

opencode

2.2. On arrive sur cette page



2.3. On lui donne un prompt de ce type

Je souhaite développer une API REST de calculatrice en Python avec FastAPI.

Fonctionnalités attendues :

Créer un endpoint POST /calculate acceptant deux nombres et une opération.

Prendre en charge l'addition, la soustraction, la multiplication et la division.

Retourner le résultat au format JSON.

Gérer les erreurs : opération inconnue, données invalides et division par zéro, avec des réponses HTTP adaptées.

Utiliser Pydantic pour valider les données entrantes.

Contraintes techniques :

Utiliser Python 3 et FastAPI avec Uvicorn.

Organiser le code proprement dans des fichiers adaptés.

Créer un fichier requirements.txt contenant les dépendances nécessaires.

Ajouter un README.md expliquant l'installation, le lancement et l'utilisation de l'API, avec des exemples de requêtes.

Respecter les bonnes pratiques de développement et produire un code lisible, maintenable et documenté.

Limites du travail :

Ne génère aucun test unitaire pour le moment. N'ajoute ni Docker ni fonctionnalités supplémentaires.

Crée directement les fichiers nécessaires dans le répertoire courant. Installe les dépendances dans un environnement virtuel Python si nécessaire, puis vérifie que l'application démarre correctement. Présente-moi les fichiers créés, les commandes pour lancer l'API et les éventuelles erreurs rencontrées.

3) Test/Vérification du fonctionnement de la calculatrice

On écrit la commande la commande :

curl -X POST http://127.0.0.1:8000/calculate -H "Content-Type: application/json" -d '{"a":5, "b":3, "operation": "add"}'

On obtient
{"a":5.0,"b":3.0,"operation":"add","result":8.0}

4) Test unitaires :

On demande à OpenCode de générer les tests unitaires.
On lance les tests :
python3 -m pytest -q
..... [100%]
========== warnings summary ==========
.venv/lib/python3.13/site-packages/fastapi/testclient.py:1
/home/gr3/Documents/API_Rest/.venv/lib/python3.13/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
28 passed, 1 warning in 0.54s

5) Analyse statique du code et scan de vulnérabilité :

On demande à OpenCode de Réaliser une analyse statique du code et un scan de vulnérabilités :

